

Session 1 – Introduction

Dr Ivan Olier

I.A.OlierCaparroso@ljmu.ac.uk

2024 Summer School in Artificial
Intelligence in Data Science

1

In this session

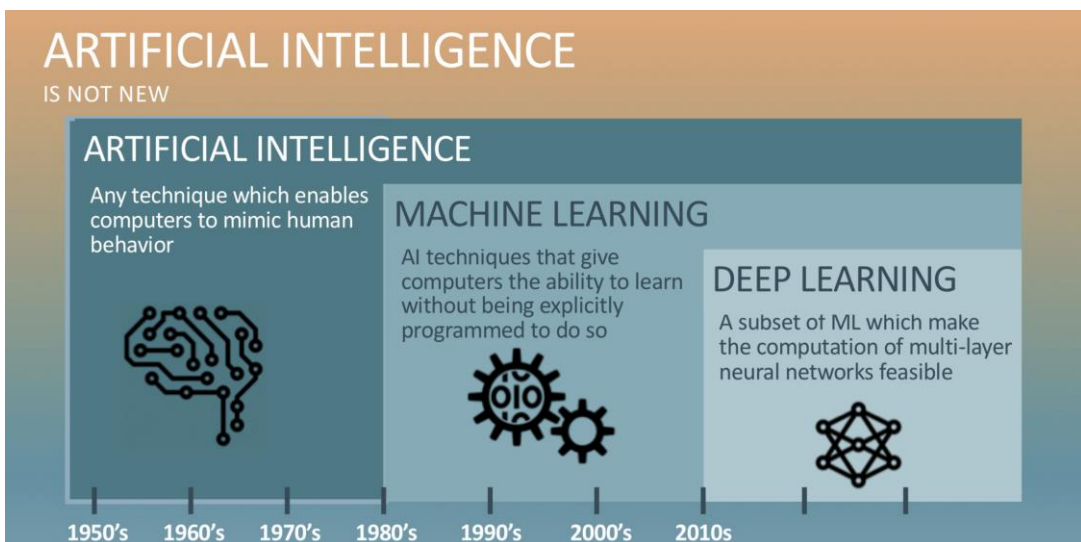
- Introduction to Artificial Intelligence (AI) in Data Science
- Artificial Neural Networks
- Hands-on activity: Introduction to Tensorflow (and Python)

2

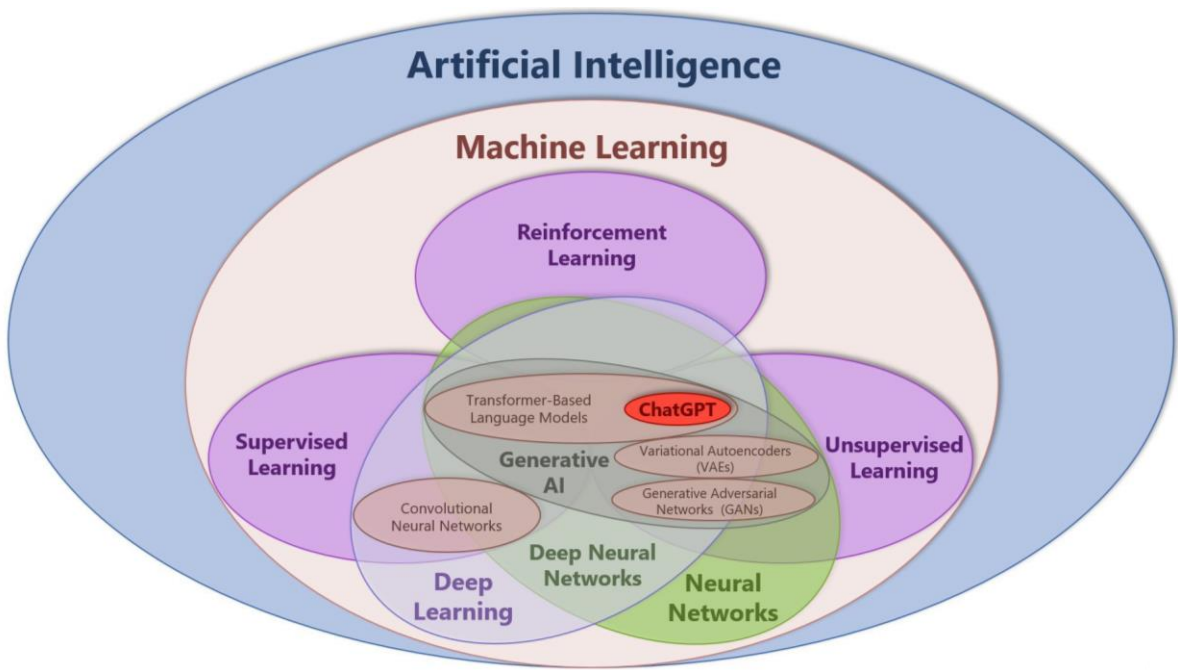
Introduction

3

Artificial Intelligence and related terms

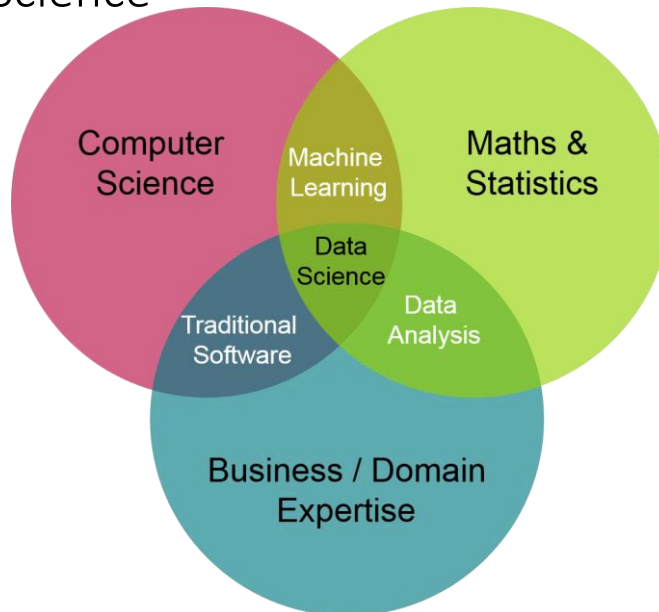


4



5

AI and Data Science



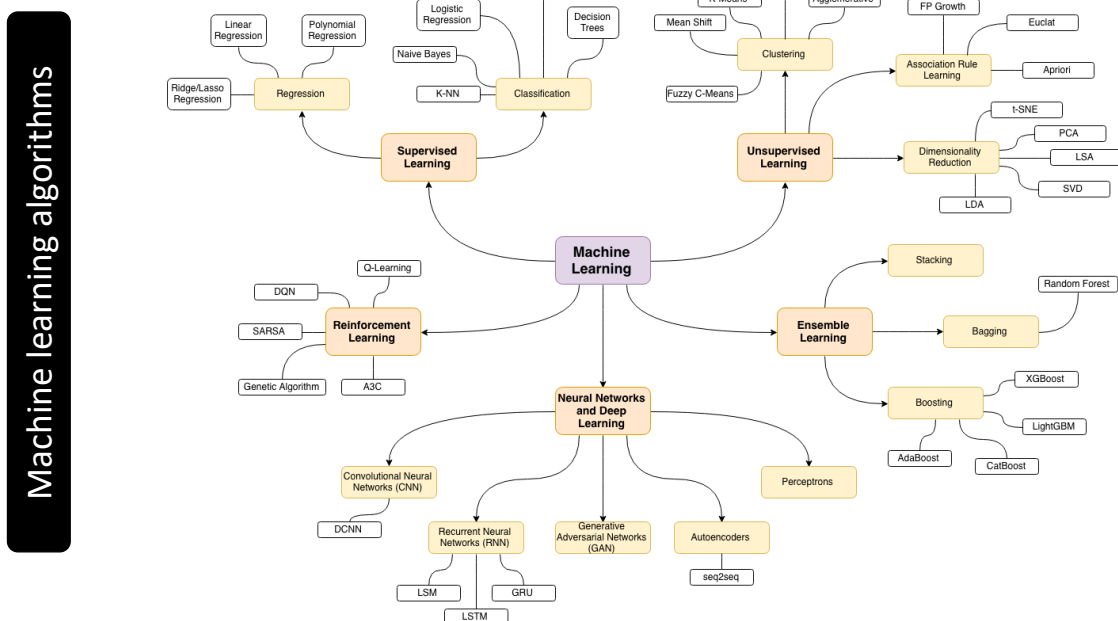
6

AI, ML, Statistics, Data Science

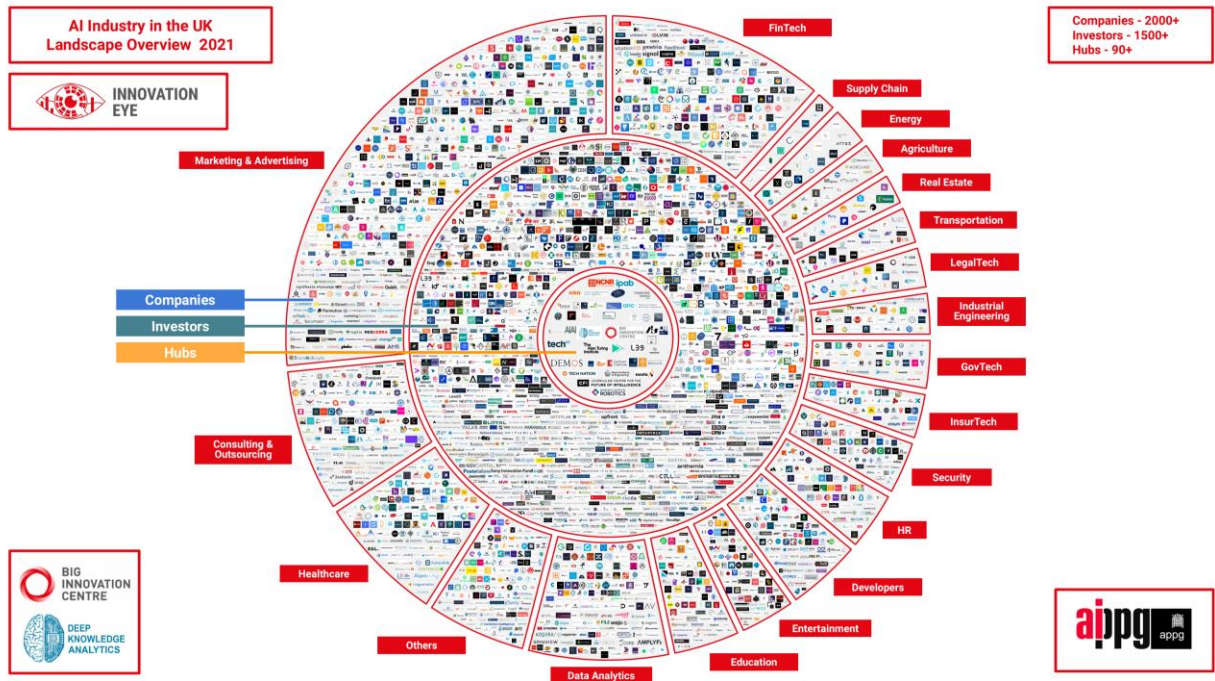
- **AI (Artificial Intelligence):** The development of computer systems capable of performing tasks that typically require human intelligence, such as speech recognition, decision-making, and problem-solving.
- **Machine Learning:** A subset of AI that involves the utilisation of algorithms and statistical models to enable computers to enhance their performance on a task without being explicitly programmed, often by learning from data.
- **Statistics:** The branch of mathematics that deals with the collection, analysis, interpretation, presentation, and organisation of data. It provides methods to draw meaningful conclusions from data and make informed decisions.
- **Data Science:** A multidisciplinary field that employs scientific methods, processes, algorithms, and systems to extract insights and knowledge from structured and unstructured data. It combines elements of statistics, machine learning, and domain expertise to uncover hidden patterns and solve complex problems.



7



8



AI in Data Science - Dr Ivan Olier

9

9

Machine learning tasks

Supervised learning

- Input and responses (outputs) are **known**, which are used to build the model. We want to use the model to predict **new** responses given new inputs.
- It is related to **predictive** modelling
- e.g. regression and classification

Unsupervised learning

- There are **no** responses. A model is built to **discover** the data structure
- It is related to **explanatory** modelling
- e.g. clustering and novelty detection

Other ML learning tasks

- Active learning, reinforcement learning, semi-supervised learning, meta-learning, transfer learning, multi-task learning, etc.

AI in Data Science - Dr Ivan Olier

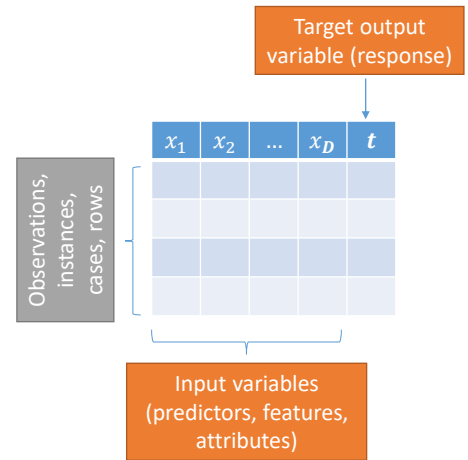
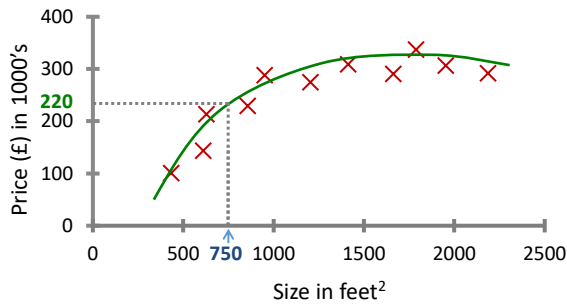
10

10

Regression

To predict **continuous valued** output as a function of the input variables

e.g. Housing price prediction as a function of house size.

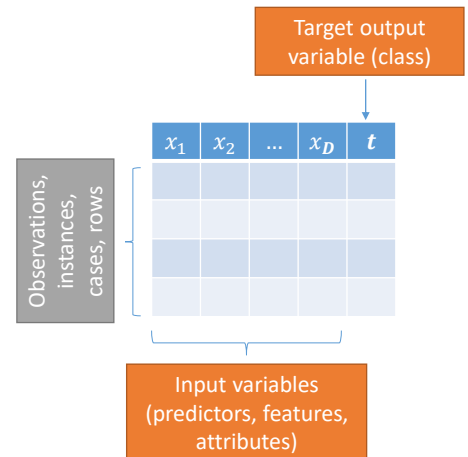
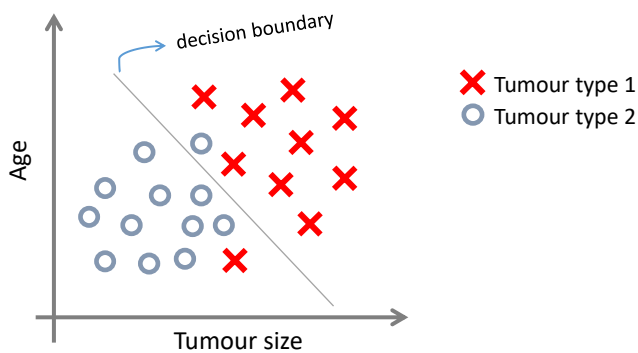


The target output is a **real** number or a whole vector of real numbers.

Classification

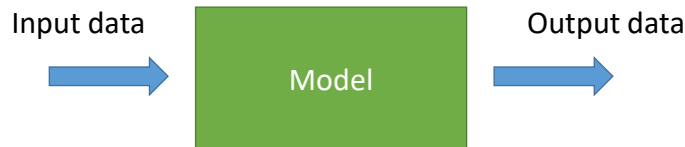
To predict **categorical** output as a function of the input variables

e.g. Tumour classification



The value of target output is a **label (or category)**

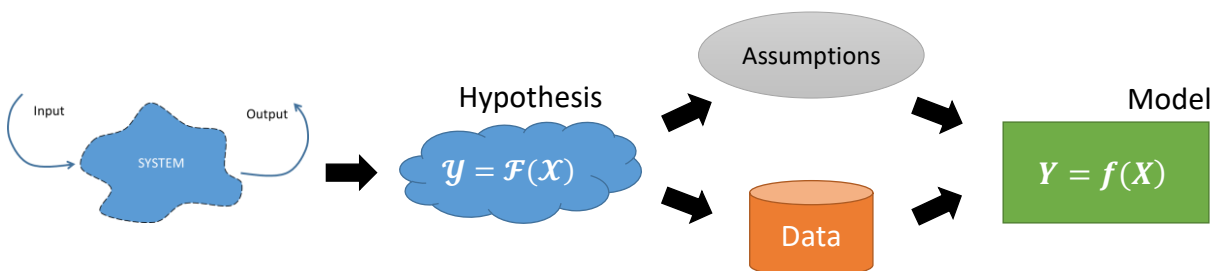
Data and models



- A **model** is a **representation** of a system using general rules and concepts.
- A **mathematical model** – Uses *mathematics* to represent the system (e.g. $y = 3x$).
- A **computer model** – Uses an *algorithm* to represent the system
(e.g. `y=function(x) 3x`)
- Signals are **collected** in the form of data (e.g. an image, audio recording, text fragment, etc)

Data-driven modelling

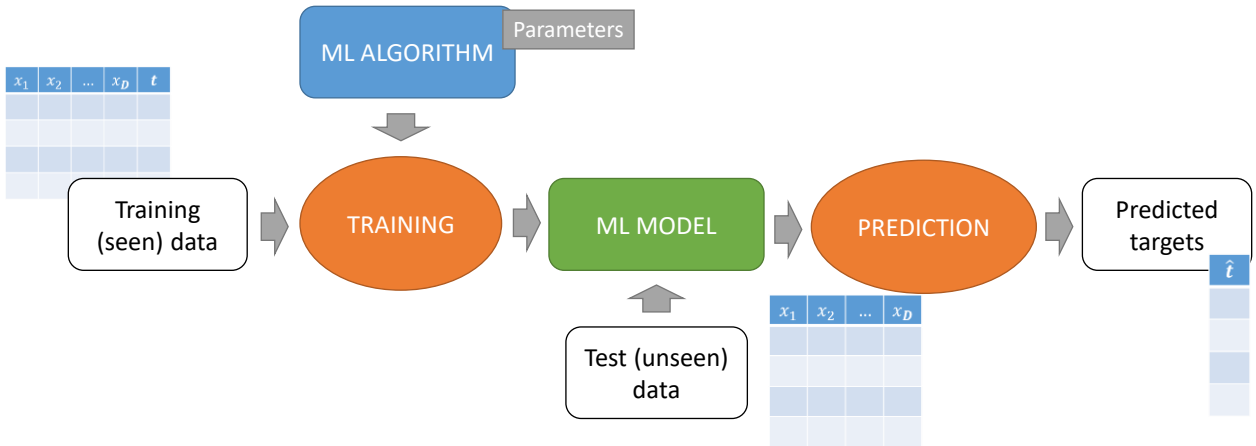
- **Modelling** is about building representations of systems.



- A **data-driven model** of a system is built by observing its input and output signals, which are collected in the form of data.
- Then, the data is used to find a set of **operations** or rules that **relates** inputs and outputs.
- Model **assumptions** are always needed (e.g. linearity, data correlation, etc). There is no free lunch! (*No free lunch theorem*, Wolpert and Macready, 1997)

Model fitting (or training)

- A ML/Statistical model is **fitted** or trained by using data (training data) to estimate its parameters.
- In turns, a fitted model can be used to make **predictions** using **unseen data** (test data).



AI in Data Science - Dr Ivan Olier

15

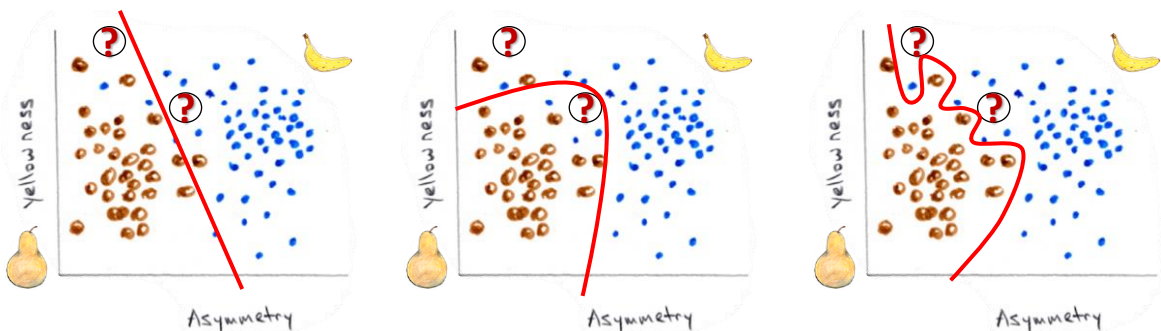
15

Model selection – classification

Model A

Model B

Model C



Model complexity

Underfitting

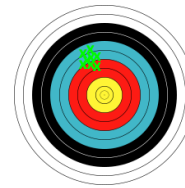
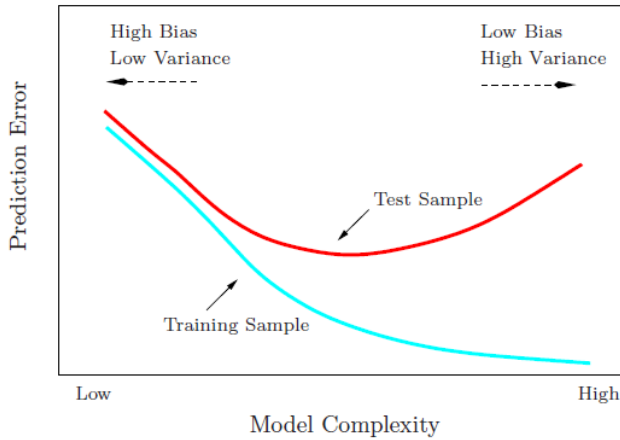
Overfitting

AI in Data Science - Dr Ivan Olier

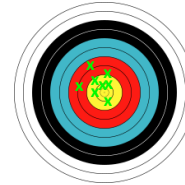
16

16

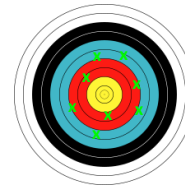
Bias – variance trade-off



High bias
Low variance



"Just right"



Low bias
High variance

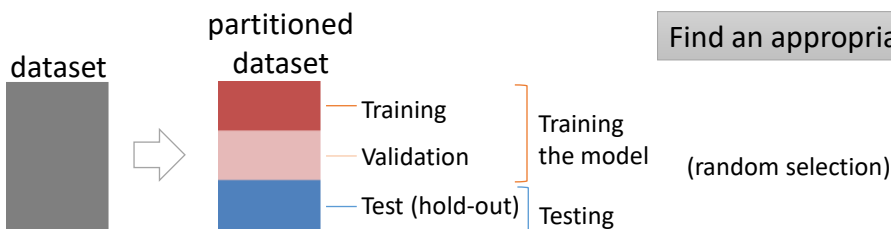
17

Model validation – predictive capabilities

– Performance with out-of-sample test sets

Considerations when partitioning datasets to training and test sets:

- More training data gives better generalisation.
- More test data gives better estimate for the classification error probability.



18

Artificial Neural Networks

19

From linear regression to neurons

Linear regression:

$$y = b + w_1x_1 + w_2x_2 + \dots + w_mx_m$$

$$v = b + \sum_{j=1}^m w_jx_j$$

$$y = v$$

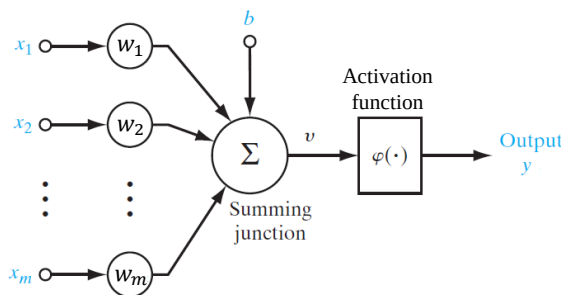
Logistic regression:

$$y = p(x) = \frac{e^{b+w_1x_1+\dots+w_mx_m}}{1 + e^{b+w_1x_1+\dots+w_mx_m}}$$

$$y = p(x) = \frac{1}{1 + e^{-(b+w_1x_1+\dots+w_mx_m)}}$$

$$y = \varphi(v) = \frac{1}{1 + e^{-v}}$$

Graphical
representation
of the models

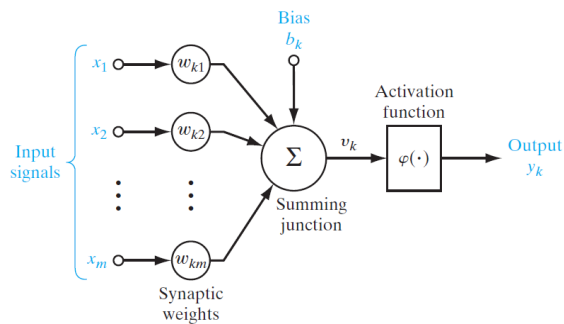


This is a **neuron** – a simple processing that performs:

1. Linear combination (adder) of inputs + bias
2. (Non)-linear activation function (φ) to produce the output.

20

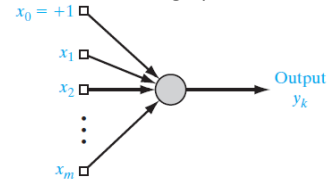
Models of neurons



$$v_k = b_k + \sum_{j=1}^m w_{kj} x_j$$

output signal of the adder

Architectural graph of a neuron



$$\begin{aligned} y_k &= \varphi(v_k) \\ &= \varphi\left(b_k + \sum_{j=1}^m w_{kj} x_j\right) \\ &= \varphi(\mathbf{w}_k^T \mathbf{x}) \end{aligned}$$

output signal of the neuron

In matrix form:

$$v_k = [b_k, w_{k1}, \dots, w_{km}] \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_j \end{bmatrix} = \mathbf{w}_k^T \mathbf{x}$$

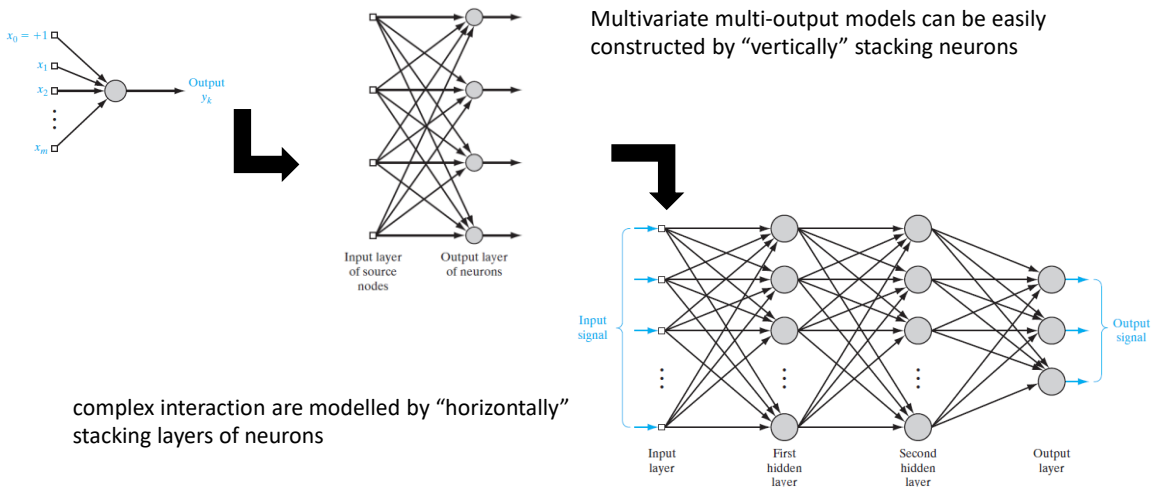
(dot product)

Summary of commonly used activation functions

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = \max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^z)$	Multi-layer Neural Networks	

From neurons to neural networks

- A neural network consists of several (could be thousands) interlinked neurons.



Definition of (artificial) neural networks

- A neural network is a **massively parallel** distributed processor made up of simple processing units that has a natural propensity for storing experiential knowledge and making it available for use.
- It **resembles** the brain in two respects:
 - Knowledge is acquired by the network from its environment through a learning process.
 - Inter-neuron connection strengths, known as synaptic **weights**, are used to store the acquired **knowledge**.

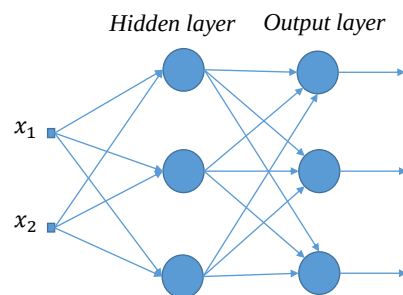
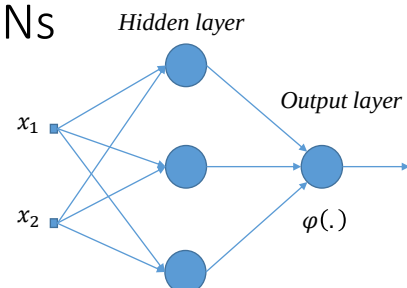
Benefits of neural networks

- inherently **nonlinear**
- the network learns from the examples by constructing an input–output mapping for the problem at hand.
- It has the capability to **adapt** their synaptic **weights** to changes in the surrounding environment.
- They can be designed to provide information not only about which particular pattern to select, but also about the confidence in the decision made.
- This may be used to reject ambiguous patterns, and thereby improve classification performance.
- **Knowledge** is represented by the very **structure** and **activation** state of a neural net.
- Every neuron in the network is potentially affected by the global activity of all other neurons in the network.
- Consequently, **contextual** information is dealt with **naturally** by a neural network.
- Neural networks can be implemented to **solve regression, classification, clustering, blind source separation, novelty detection**, amongst other kind of **data mining** problems.

Regression and classification with NNs

- $\varphi(\cdot)$ is **linear** for regression tasks (e.g. $\varphi(v) = v$)
- $\varphi(\cdot)$ is either **sigmoid**, **tanh**, or equivalent, for a two-class classification task.
- A multi-output regression task is just an **extension** of the same architecture.
- It is also called “**multi-task learning**”.
- This network can also be used for multi-class classification tasks.
- The **softmax** activation function is typically used. It is seen as a **generalised** sigmoid, so the outputs are still probabilities:

$$y_i = \frac{e^{v_i}}{\sum_{i=1}^c e^{v_i}}$$



Gradient descent

- If we define the loss (or error) as a function of the weights, $\mathcal{L}(\mathbf{w})$, it makes sense to let the weights update proportionally to loss change:

$$\Delta \mathbf{w}^{(n)} \propto \frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(n)}}$$

where $\Delta \mathbf{w}^{(n)} = \mathbf{w}(n+1) - \mathbf{w}(n)$, and n labels the iteration step, so:

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(n)}}, \quad \text{where } \eta > 0 \text{ is the } \textit{learning rate}.$$

- After each such update, the gradient is **re-evaluated** for the new weight vector and the process repeated.
- Note that the loss function is defined with respect to a training set, and so each step requires that the entire training set be processed in order to evaluate the gradient.
- The gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(n)}}$ is estimated using the **back-propagation** algorithm.

Hands-On Activity

Introduction to Programming with Python

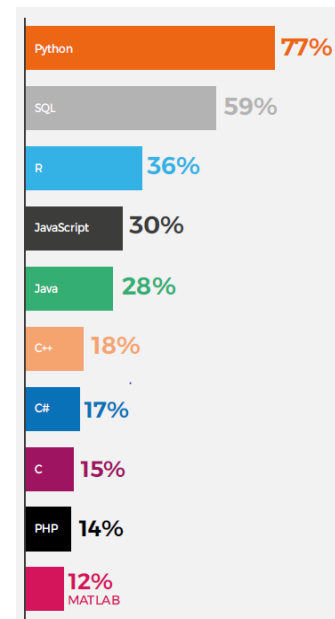
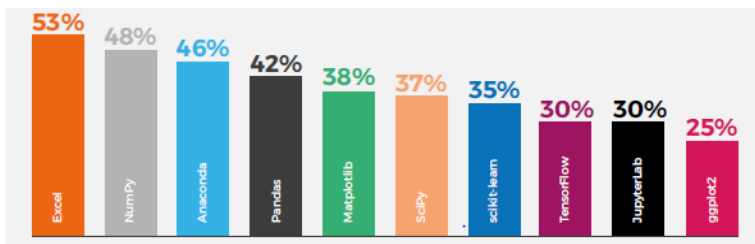
Python

- Python can be used for a variety of different applications such as web development, software development, scripting, machine learning and performing complex mathematics.
- Python can be used to construct production ready end to end pipelines.
- Python enjoys cross platform support (Windows, Mac, Linux, Raspberry Pi, etc.).
- The most recent major version of Python is Python 3 (which is what we will be using during this module).
- A good resource for python is python.org
 - <https://www.python.org/about/gettingstarted/>



Why Python for Machine Learning?

- Python is the most widely used programming language by machine learning scientists.
- Python is easier to learn than other programming languages such as R
- Eight of the top ten most used data tools are derived from or utilise Python.



Jupyter Notebooks

- One of the most common ways to write and execute Python code is through the use of a Jupyter Notebooks which are run directly in the browser.
- They are also useful for making notes through the use of markdown and act as executable documents and should be considered as a recipe for your data science task.
- One advantage of using Jupyter Notebooks is that code can be executed directly in the browser allowing you to instantly see the results.
- Notebooks can be exported as executable .py files which can be used in your development pipeline.
- Notebooks ensure reproducibility by easily sharing the notebook and associated data.



31

Jupyter Notebooks

jupyter Optical Coherence Tomography-Copy1 Last Checkpoint: Last Sunday at 6:14 PM (autosaved)

File Edit View Insert Cell Kernel Navigate Widgets LaTeX Envs Help

Run Code

Contents

- 1.1 Optical Coherence Tomography
- 1.2 Imports, preliminaries, def
- 1.3 Imaging system - overview
- 1.3 OCT Theory - overview
- 1.3.1 Comments and calcula
- 1.3.1.1 Resolution "back-of
- 1.3.1.2 Scan depth "back-of
- 1.3.1.3 Scaling of coheren
- 1.3.2 Time Domain OCT (TC
- 1.3.2.1 Detection-bandwid
- 1.3.2.2 TDOCT SNR and
- 1.3.3 Fourier Domain OCT (FDOCT)
- 1.3.3.1 Impact of finite spe
- 1.3.3.2 Interleave: Finite sa
- 1.3.3.3 Impact of finite nur
- 1.3.3.4 FDOCT SNR and
- 1.3.4 Spectral domain/swept
- 1.3.4.1 SDOCT SNR and i
- 1.4 Simulation
- 1.5 Potential laser sources

1.3.3 Fourier Domain OCT (FDOCT)

In FDOCT, the different wavelengths are collected on a spectrometer, with N_{det} pixels, and spectral resolution $\delta\lambda$.

Returning again to Eq. (8) (see, e.g., Izatt and Choma (Izatt J.A., Choma M.A. (2008) Theory of Optical Coherence Tomography. In: Drexler W., Fujimoto J.G. (eds) Optical Coherence Tomography: Biological and Medical Physics, Biomedical Engineering. Springer, Berlin, Heidelberg. doi: https://doi.org/10.1007/978-3-540-77550-5_2; alternate link: https://www.researchgate.net/publication/226173102_Theory_of_Optical_Coherence_Tomography/download).

$$I_D(k) = \frac{Q}{4} S(k) \left[R_R + \sum_{n=1}^N R_n \right] \quad \text{"DC terms"}$$

$$+ \frac{Q}{2} S(k) \left[\sum_{n=1}^N \sqrt{R_R R_n} \cos[2k(z_R - z_n)] \right] \quad \text{"Cross-correlation terms"}$$

$$+ \frac{Q}{2} S(k) \left[\sum_{n=1}^N \sqrt{R_n R_m} \cos[2k(z_n - z_m)] \right] \quad \text{"Autocorrelation terms"}$$

In the FDOCT configuration, z_R is held fixed.

```
In [23]: lambda_0 = 1.5500
k_0 = 2.0*np.pi/lambda_0
Dlambda_0 = 0.100
Dk = 2.0*np.pi*Dlambda_0/lambda_0**2.0
k_range = np.linspace(-3.0*Dk+k_0, +3.0*Dk+k_0, 10000)

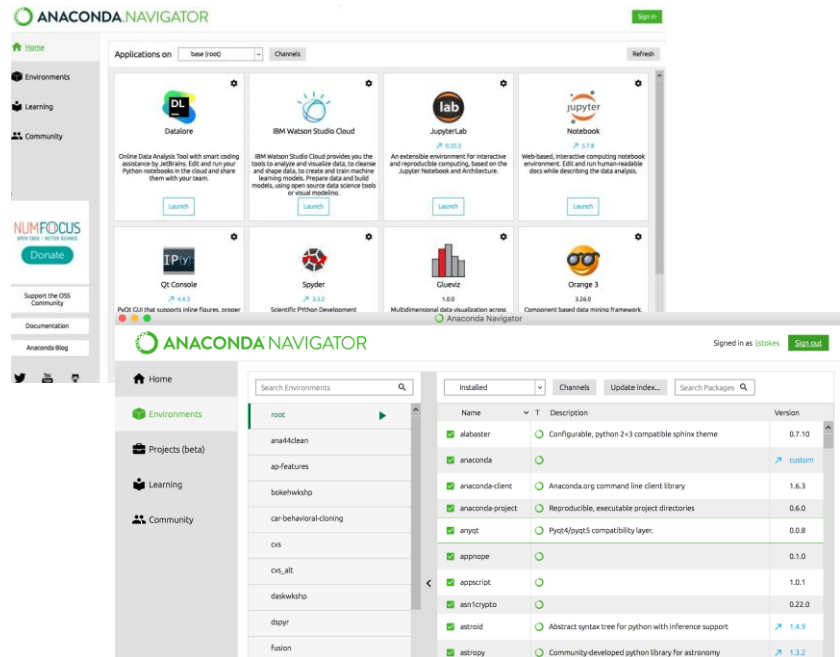
TD_OCT_signal = 0.25*0.5*(np.exp(-((k_range - k_0)/Dk)**2.0) \
+ 0.5*np.sqrt(0.5*(2.0-4))*(np.exp(-((k_range - k_0)/Dk)**2.0) \
*np.cos(2.0*k_range*(50.0)) \
+ 0.5*np.sqrt(0.5*(1.5E-4))*(np.exp(-((k_range - k_0)/Dk)**2.0) \
*np.cos(2.0*k_range*(200.0))
```

```
In [26]: fig_disp
Out[26]:
```

32

Anaconda

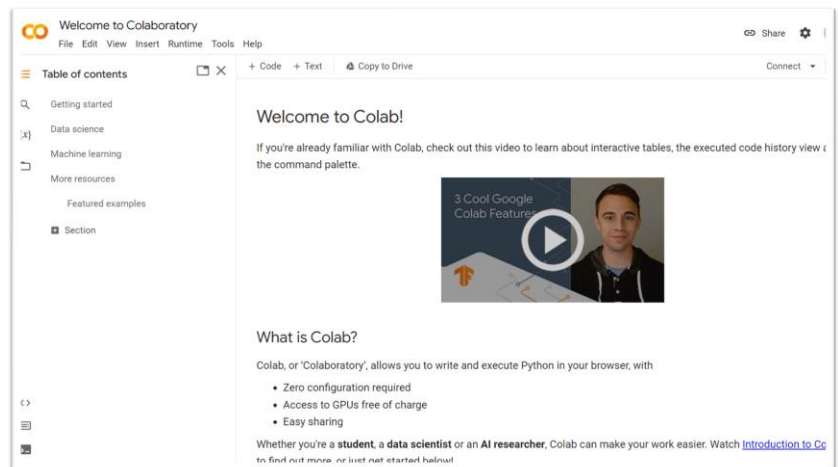
- Anaconda is a scalable data science platform which allows you to combine but also segregate different packages and frameworks to prove canned environments.
- As these environments are isolated from each other they can be safely modified and changed without affecting any other environments used.



33

Google Colaboratory

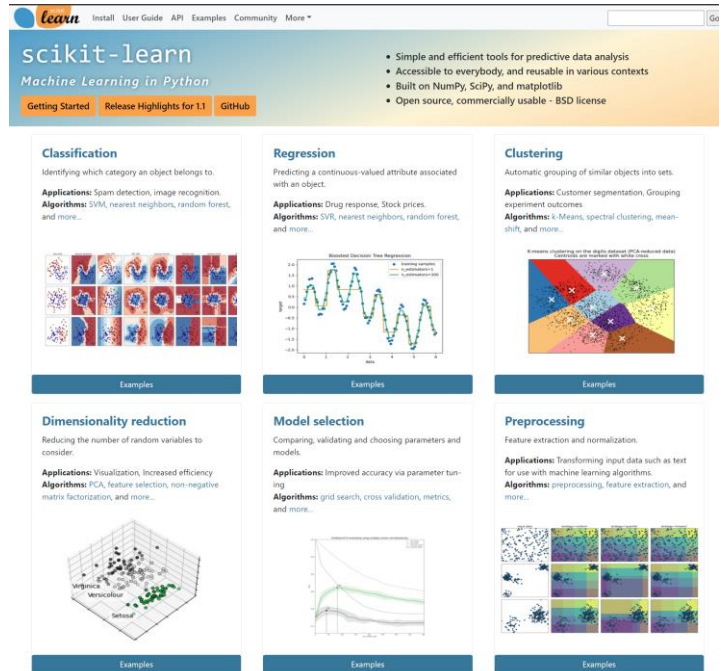
- **A free Jupyter notebook environment:**
 - Google Colaboratory is a free, Jupyter notebook environment that runs in the cloud.
- **Shareable and reproducible:**
 - Colab notebooks can be shared with others and are fully reproducible.
- **Access powerful computing resources:**
 - Colab notebooks can access powerful computing resources, such as GPUs and TPUs, for free.



34

Scikit-learn

- For machine learning tasks this module will introduce you one of the most popular Python machine learning libraries called Scikit-learn.



Other useful Python libraries

- NumPy** – operations on multi-dimensional arrays and matrices
- SciPy** – used for scientific and technical computing
- Matplotlib** – standard but powerful plotting library
- Pandas** – data manipulation and analysis
- Seaborn** – high-level interface of matplotlib
- Plotly** – for interacting, publication-quality graphs
- Statsmodels** – statistical package



Deep learning frameworks

- The explosion of DL successes has been attributed to the supporting frameworks provided by the biggest global technology companies such as Google, Facebook and Microsoft.
- These companies are at the forefront of applied AI and its adoption within many business sectors.
- They have developed and offer DL frameworks that are free to use which have significantly eased to training and hosting of models.



MNIST dataset

- The MNIST database (Modified National Institute of Standards and Technology database) is a large database of handwritten digits that is commonly used for training various image processing systems.
- It contains 60,000 training images and 10,000 testing images.
- In the original paper, LeCun et al. (1998)*, a support vector machine was used to get an **error rate of 0.8%**

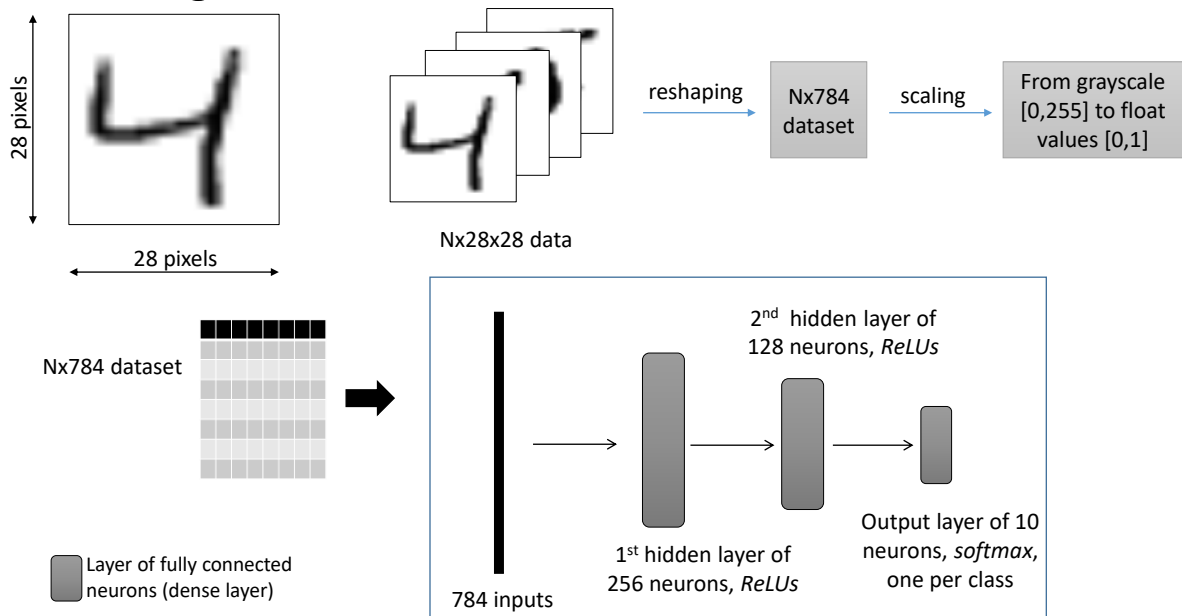


* LeCun, Yann; Léon Bottou; Yoshua Bengio; Patrick Haffner (1998). "Gradient-Based Learning Applied to Document Recognition" (PDF). Proceedings of the IEEE.

Type	Classifier	Error rate (%)
Linear classifier	Pairwise linear classifier	7.6 ^[9]
Non-linear classifier	40 PCA + quadratic classifier	3.3 ^[9]
Random Forest	Fast Unified Random Forests for Survival, Regression, and Classification (RF-SRC) ^[23]	2.8 ^[24]
Deep neural network (DNN)	2-layer 784-800-10	1.6 ^[26]
Boosted Stumps	Product of stumps on Haar features	0.87 ^[22]
Deep neural network	2-layer 784-800-10	0.7 ^[26]
Support-vector machine (SVM)	Virtual SVM , deg-9 poly, 2-pixel jittered	0.56 ^[25]
K-Nearest Neighbors	K-NN with non-linear deformation (P2DHMDM)	0.52 ^[21]
Deep neural network	6-layer 784-2500-2000-1500-1000-500-10	0.35 ^[27]
Convolutional neural network (CNN)	6-layer 784-40-80-500-1000-2000-10	0.31 ^[15]
Convolutional neural network	6-layer 784-50-100-500-1000-10-10	0.27 ^[28]
Convolutional neural network	Committee of 35 CNNs, 1-20-P-40-P-150-10	0.23 ^[8]
Convolutional neural network	Committee of 5 CNNs, 6-layer 784-50-100-500-1000-10-10	0.21 ^[1,27]
Random Multimodel Deep Learning (RMDL)	10 NN-10 RNN - 10 CNN	0.18 ^[19]
Convolutional neural network	Committee of 20 CNNs with Squeeze-and-Excitation Networks ^[29]	0.17 ^[29]

Some ML methods used with the MNIST database

Modelling the MNIST data with Tensorflow



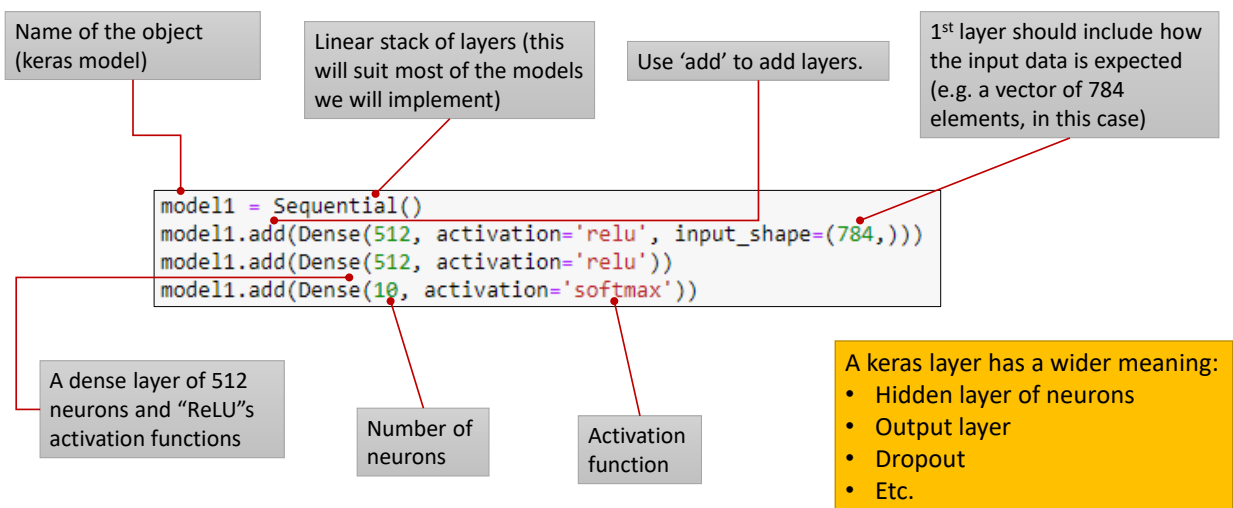
AI in Data Science - Dr Ivan Olier

39

39

Modelling the MNIST data with Tensorflow

Step 1) To build the architecture



AI in Data Science - Dr Ivan Olier

40

40

Modelling the MNIST data with Tensorflow

The keras object is easy to read:

Sequential model

```
model1.summary()
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	401920
dense_1 (Dense)	(None, 512)	262656
dense_2 (Dense)	(None, 10)	5130
Total params: 669,706		
Trainable params: 669,706		
Non-trainable params: 0		

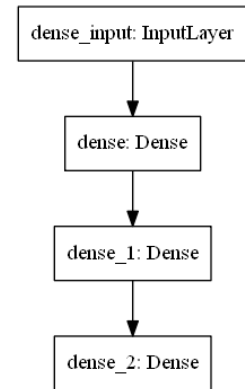
512 x 784 connections (weights) + 512 biases = 401,920

Layer name (you can change this too)

You might have pre-trained layers (more on this later)

Batch size (none in our case)

The architecture can be visualised:



Modelling the MNIST data with Tensorflow

Step 2) To set the optimiser

Loss function – the one that keras will use to assess convergence. More details in <https://keras.io/api/losses/>

```
model1.compile(
    loss='categorical_crossentropy',
    metrics=['accuracy'],
    optimizer='adam'
)
```

Metrics – not used to assess convergence

Optimisers :

- RMSprop - <https://keras.io/api/optimizers/rmsprop>
- SGD - <https://keras.io/api/optimizers/sgd>
- Adam - <https://keras.io/api/optimizers/adam>
- More? See Keras Reference : <https://keras.io/>

Modelling the MNIST data with Tensorflow

Step 3) To train the model

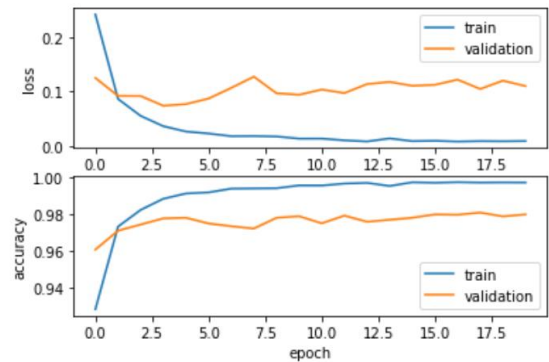
```
history = model1.fit(X_train, y_train,
                    batch_size=128, epochs=20,
                    verbose=1,
                    validation_split=0.2)
```

Training data

You can let Keras set aside a validation set.

The train model is in "model1". You can optionally record training details in "history"

plot_history(history)



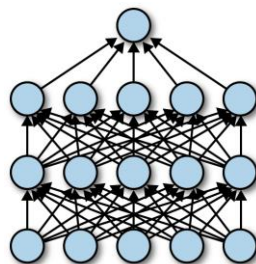
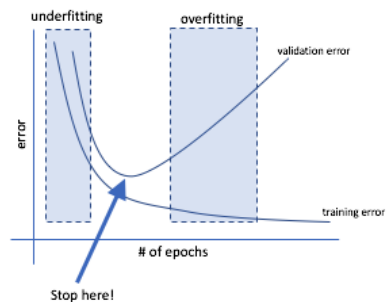
We may have overfitting !!!



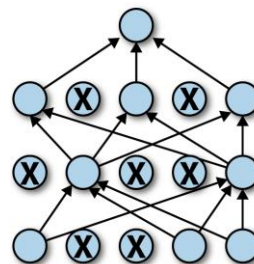
Ways to reduce overfitting

A large number of different methods have been developed:

- **Weight-decay** – where you try and keep the weights of the networks **small**. It will make the model **simpler**.
- **Early stopping** – where you try to stop the iterative training process when validation error starts increasing.
- **Dropout** – where you try and make your model more robust by randomly **dropping out** hidden units when you're training it.



(a) Standard Neural Net



(b) After applying dropout

Modelling the MNIST data with Tensorflow

Step 1B) To add dropout layers

```
from tensorflow.keras.layers import Dropout

model2 = Sequential()
model2.add(Dense(512, activation='relu', input_shape=(784,)))
model2.add(Dropout(rate=0.4))
model2.add(Dense(512, activation='relu'))
model2.add(Dropout(rate=0.4))
model2.add(Dense(10, activation='softmax'))

model2.summary()
```

Dropout layers – “rate” is the dropout out rate [0, 1]

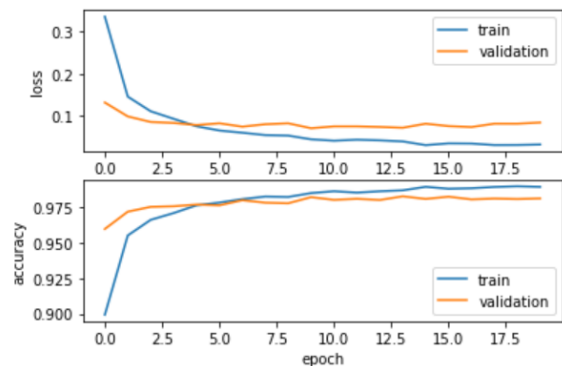
Modelling the MNIST data with Tensorflow

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 512)	401920
dropout (Dropout)	(None, 512)	0
dense_4 (Dense)	(None, 512)	262656
dropout_1 (Dropout)	(None, 512)	0
dense_5 (Dense)	(None, 10)	5130
Total params: 669,706		
Trainable params: 669,706		
Non-trainable params: 0		

```
test_scores_model2 = model2.evaluate(X_test, y_test, verbose=0)
print("Test loss model 2:", test_scores_model2[0])
print("Test accuracy model 2:", test_scores_model2[1])
train_scores_model2 = model2.evaluate(X_train, y_train, verbose=0)
print("Train loss model 2:", train_scores_model2[0])
print("Train accuracy model 2:", train_scores_model2[1])
```

```
Test loss model 2: 0.0646570548415184
Test accuracy model 2: 0.98489997111816
Train loss model 2: 0.02108081988990307
Train accuracy model 2: 0.9950500130653381
```



- The overfitting level is **now reduced** in the new model
- We can also set aside a test set (for instance if we are performing **out-of-sample** validation)

<https://keras.rstudio.com/index.html>